

```

import pandas as pd import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

df = pd.read_csv("weather_activity_dataset.csv")

df

```

	Weather	Activity
0	Cloudy	Go Out
1	Windy	Go Out
2	Sunny	Go Out
3	Sunny	Go Out
4	Sunny	Go Out
..	...	...
195	Rainy	Stay In
196	Windy	Stay In
197	Sunny	Go Out
198	Sunny	Go Out
199	Windy	Go Out

[200 rows x 2 columns]

```
df.head()
```

	Weather	Activity
0	Cloudy	Go Out
1	Windy	Go Out
2	Sunny	Go Out
3	Sunny	Go Out
4	Sunny	Go Out

```
df.tail()
```

	Weather	Activity
195	Rainy	Stay In
196	Windy	Stay In
197	Sunny	Go Out
198	Sunny	Go Out
199	Windy	Go Out

```

df.shape

(200, 2)

df.info()

<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Weather     200 non-null    str
1   Activity    200 non-null    str
dtypes: str(2)
memory usage: 3.3 KB

df.isnull().sum()

Weather      0
Activity     0
dtype: int64

df.describe(include='all')

      Weather Activity
count      200      200
unique        4        2
top      Windy   Go Out
freq       55     107

df.duplicated().sum()

np.int64(192)

df["Weather"].unique()

<StringArray>
['Cloudy', 'Windy', 'Sunny', 'Rainy']
Length: 4, dtype: str

df["Activity"].unique()

<StringArray>
['Go Out', 'Stay In']
Length: 2, dtype: str

weather_encoder = LabelEncoder()

df["Weather"] = weather_encoder.fit_transform(df["Weather"])

df.head()

```

	Weather	Activity
0	0	Go Out
1	3	Go Out
2	2	Go Out
3	2	Go Out
4	2	Go Out

```
activity_encoder = LabelEncoder()
```

```
df["Activity"] = activity_encoder.fit_transform(df["Activity"])
```

```
df.head()
```

	Weather	Activity
0	0	0
1	3	0
2	2	0
3	2	0
4	2	0

```
df.corr()
```

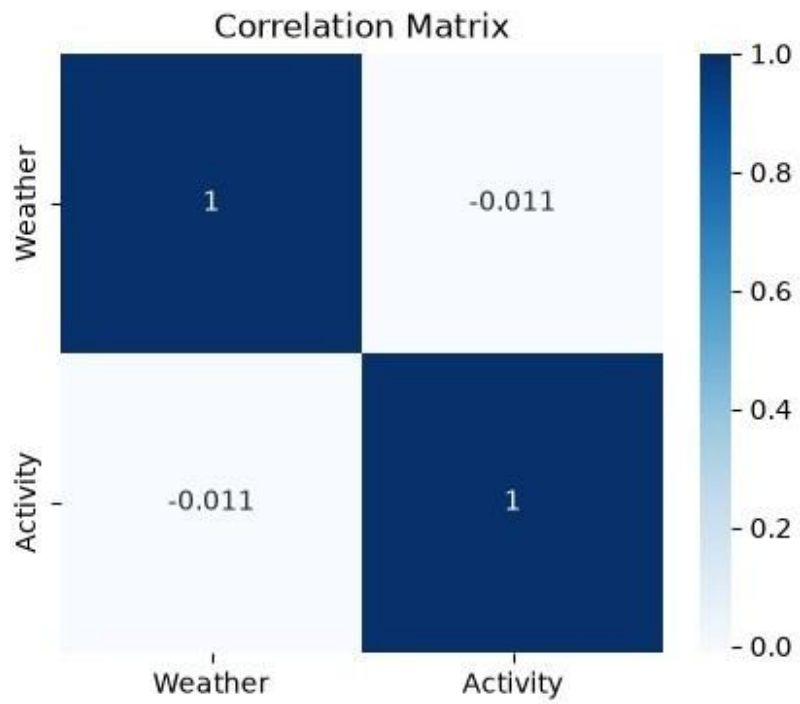
	Weather	Activity
Weather	1.000000	-0.010905
Activity	-0.010905	1.000000

```
plt.figure(figsize=(5,4))
```

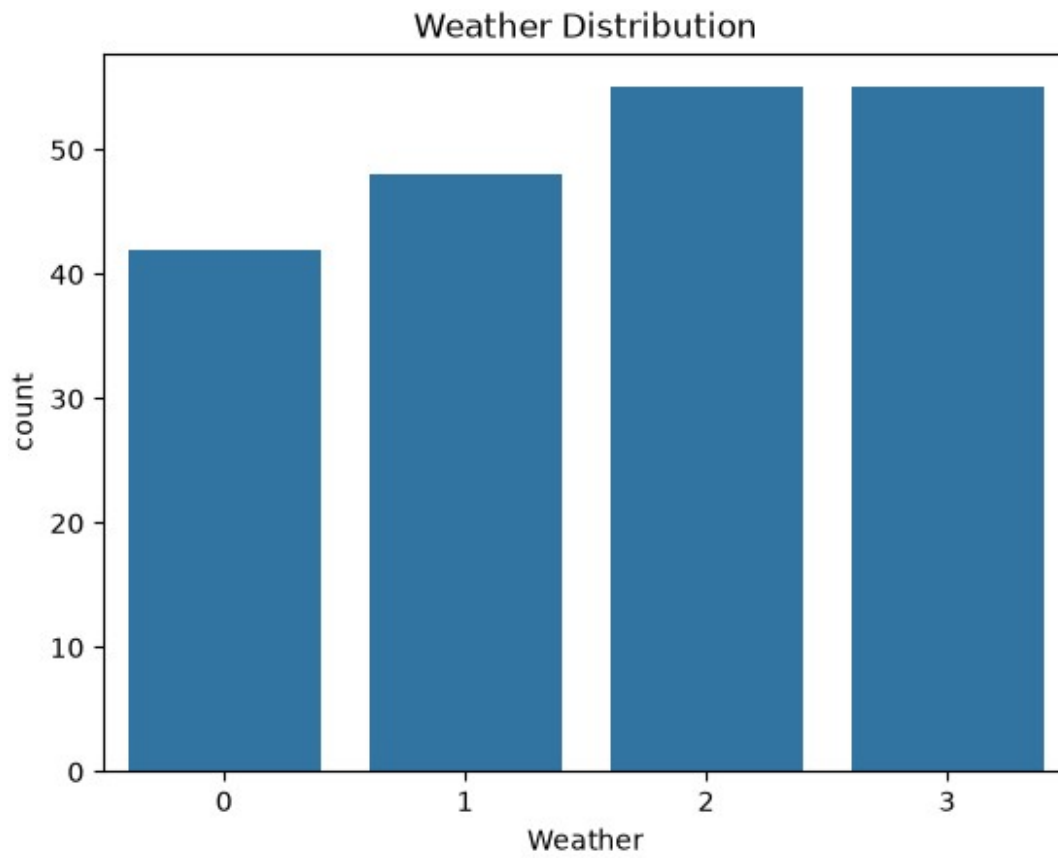
```
sns.heatmap(df.corr(),
             annot=True,
             cmap="Blues")
```

```
plt.title("Correlation Matrix")
```

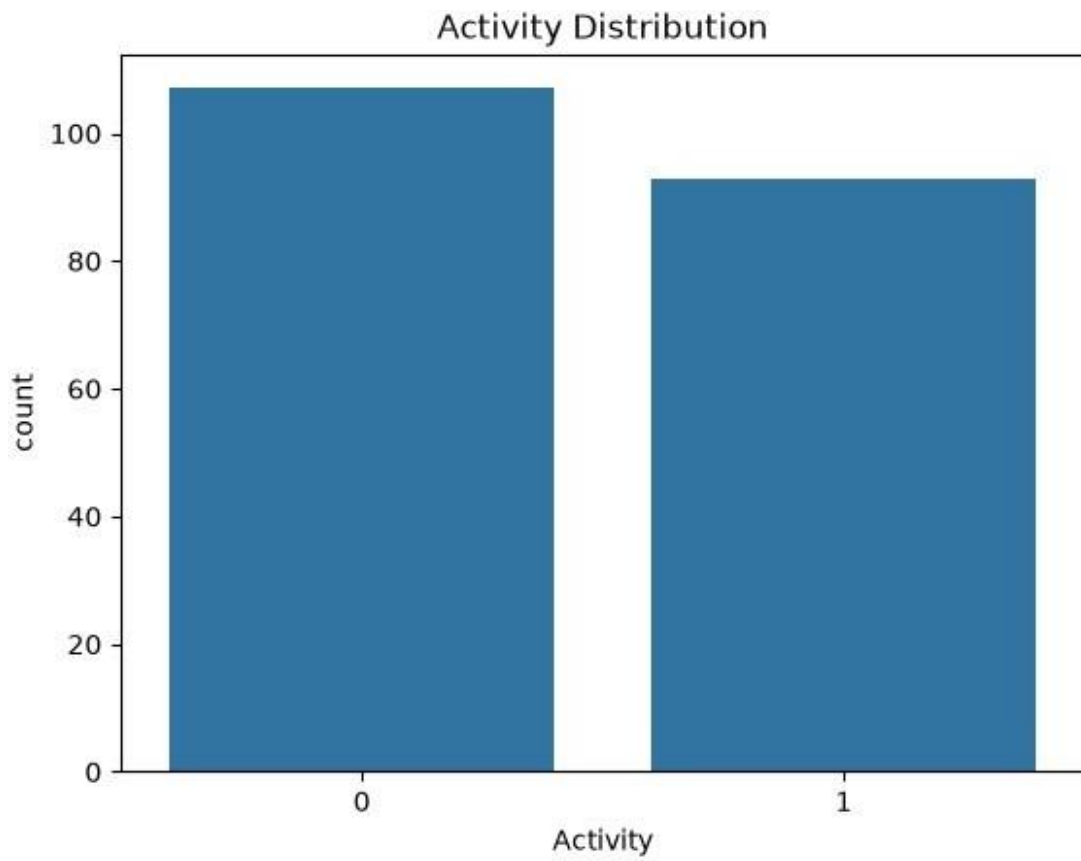
```
plt.show()
```



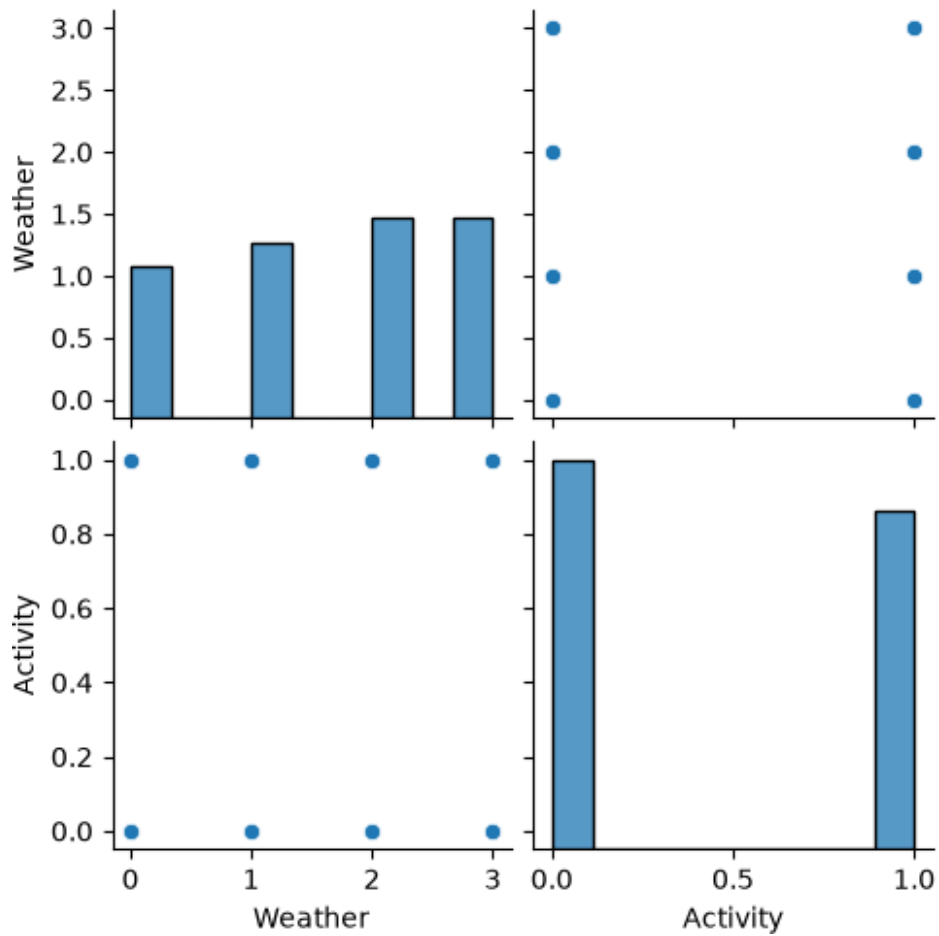
```
sns.countplot(x="Weather",data=df)
plt.title("Weather Distribution")
plt.show()
```



```
sns.countplot(x="Activity",data=df)
plt.title("Activity Distribution")
plt.show()
```



```
sns.pairplot(df)  
plt.show()
```



```
X = df[["Weather"]]
```

```
y = df["Activity"]
```

```
X.head()
```

	Weather
0	0
1	3
2	2
3	2
4	2

```
y.head()
```

0	0
1	0
2	0
3	0
4	0

```
Name: Activity, dtype: int64
```

```

X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print(X_train.shape)

print(X_test.shape)

print(y_train.shape)

print(y_test.shape)

(160, 1)
(40, 1)
(160,)
(40,)

model = DecisionTreeClassifier(random_state=42)

model.fit(X_train,y_train)

DecisionTreeClassifier(random_state=42)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test,y_pred)

print("Accuracy =",accuracy)

Accuracy = 0.625

cm = confusion_matrix(y_test,y_pred)

print(cm)

[[11 10]
 [ 5 14]]

plt.figure(figsize=(5,4))

sns.heatmap(cm,
            annot=True,
            fmt="d",
            cmap="Greens")

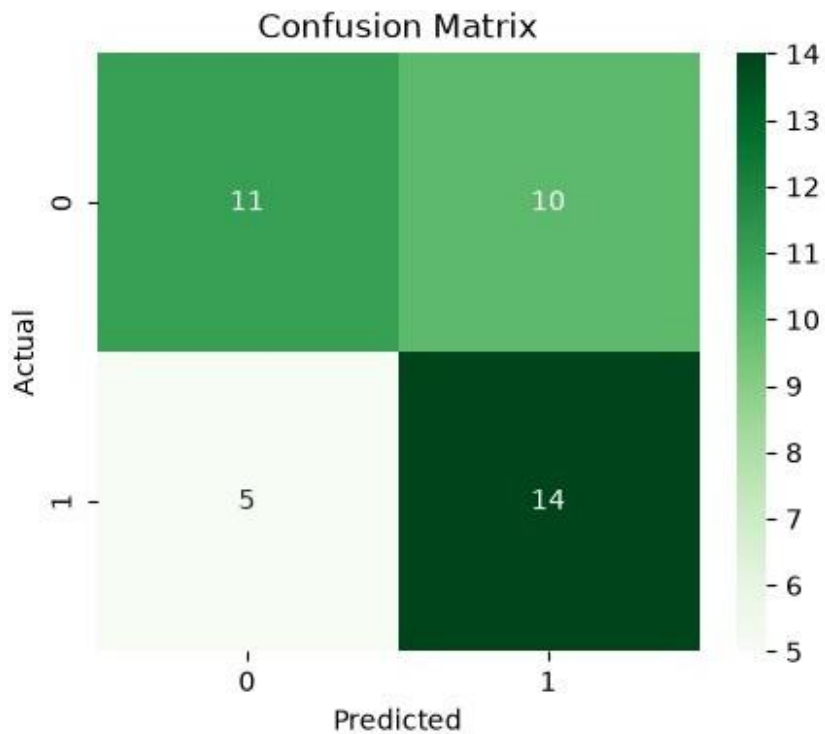
plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

```

```
plt.show()
```



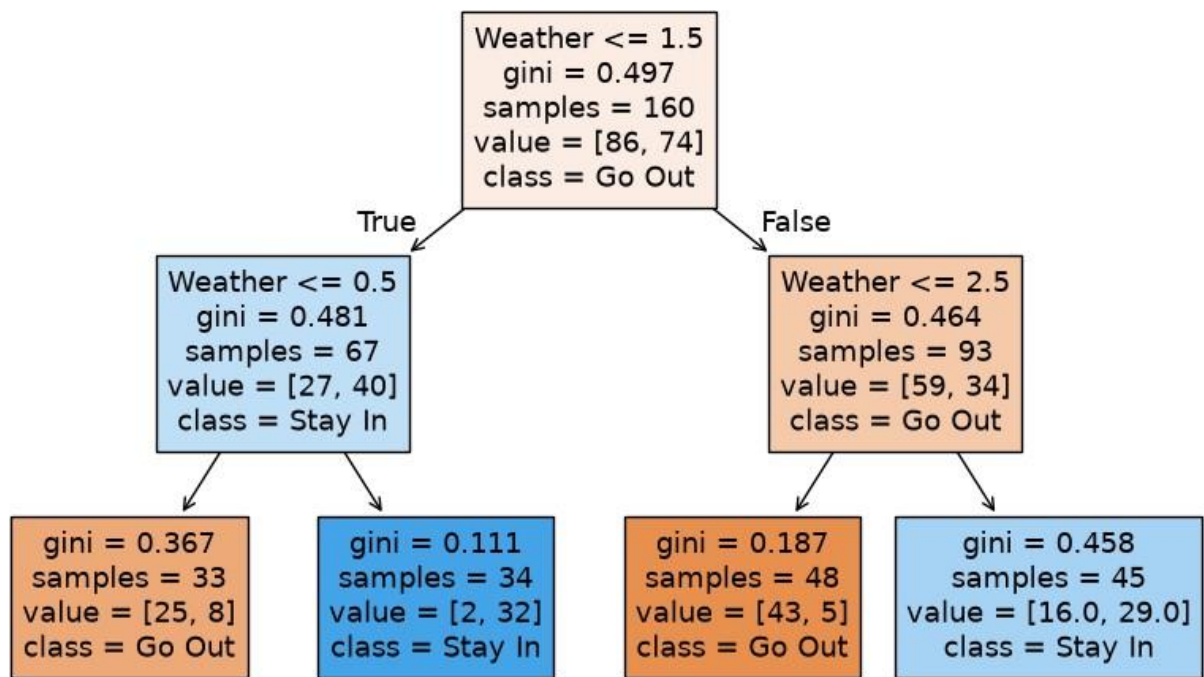
```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.69	0.52	0.59	21
1	0.58	0.74	0.65	19
accuracy			0.62	40
macro avg	0.64	0.63	0.62	40
weighted avg	0.64	0.62	0.62	40

```
plt.figure(figsize=(10,6))
```

```
plot_tree(model,
           feature_names=["Weather"],
           class_names=activity_encoder.classes_,
           filled=True)
```

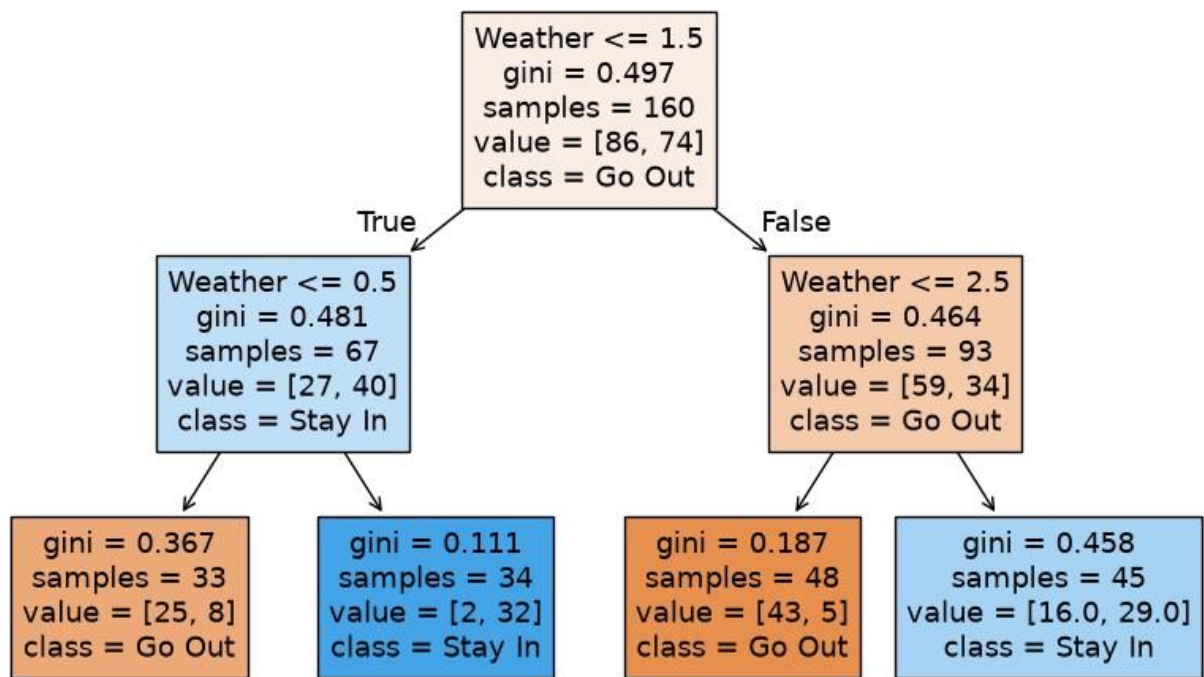
```
plt.show()
```



```
plt.figure(figsize=(10,6))

plot_tree(model,
          feature_names=["Weather"],
          class_names=activity_encoder.classes_,
          filled=True)

plt.show()
```



```

new_weather = pd.DataFrame({
    "Weather": [weather_encoder.transform(["Sunny"])[0]]
})

prediction = model.predict(new_weather)
print("Prediction Code:", prediction)
Prediction Code: [0]

new_weather = pd.DataFrame({
    "Weather": [weather_encoder.transform(["Sunny"])[0]]
})

prediction = model.predict(new_weather)
print("Prediction Code:", prediction)
Prediction Code: [0]

probability = model.predict_proba(new_weather)
print(probability)
[[0.89583333 0.10416667]]

results = pd.DataFrame({
    "Actual": y_test,

```

```

        "Predicted": y_pred
    })

results.head()

```

	Actual	Predicted
95	0	0
15	0	1
30	1	0
158	1	1
128	0	1

```

results["Actual"] =
activity_encoder.inverse_transform(results["Actual"])
results["Predicted"] =
activity_encoder.inverse_transform(results["Predicted"])

results.head()

```

	Actual	Predicted
95	Go Out	Go Out
15	Go Out	Stay In
30	Stay In	Go Out
158	Stay In	Stay In
128	Go Out	Stay In

```

results.to_csv("Weather_Activity_Predictions.csv", index=False)

print("Prediction file saved successfully.")

Prediction file saved successfully.

```